RWANDA POLYTECHNIC

KIGALI COLLEGE

**ACADEMIC YEAR: 2025-2026**

**DEPARTMENT: EEE, PROGRAM: ETT**

**RTQF Level, Year: BTech, YEAR 4**

**MODULE CODE & TITLE: GENML801- MACHINE LEARNING**

**SEMESTER: 1**

**GROUP NAMES:**


1. **MUGISHA EMMANUUEL 20RP00370**
2. **NDAYIMENYE SAMUEL 26RP02292**
3. **NSABIMANA RUKUNDO OLIVIEL 26RP01994**
4. **IRABISHOBOYE GILBERT 26RP00139**


**Date: 1st November 1, 2026**

# FRUIT DETECTION SYSTEM USING YOLOv8 AND STREAMLIT

## 1. Introduction

Fruit detection is an important application of computer vision and artificial intelligence. It can be used in agriculture, fruit sorting, quality inspection, and automated inventory management. Traditional fruit identification methods often require manual inspection, which can be time-consuming and may produce inconsistent results.

This project developed a fruit detection system using the YOLOv8 object detection algorithm. The system was designed to identify three fruit classes: apples, bananas, and oranges. It uses a trained deep learning model to detect fruits in uploaded images, classify them, and display bounding boxes around detected objects.

The application was developed using Python and Streamlit, while the model was trained and evaluated using Google Colab. GitHub was used for source-code and model-file management, and Streamlit Cloud was used to deploy the application online.

## 2. Project Objectives

The main objective was to develop an AI-based application capable of detecting and identifying apples, bananas, and oranges from images.

The specific objectives were:

1. To prepare and validate a fruit image dataset containing apples, bananas, and oranges.
2. To train and fine-tune a YOLOv8 object detection model.
3. To evaluate the model using validation and test datasets.
4. To develop a user-friendly web application using Streamlit.
5. To integrate the trained model into the web application.
6. To publish the application online using GitHub and Streamlit Cloud.
7. To test the system using individual-fruit and mixed-fruit images.

## 3. Technologies and Tools Used

| **Technology** | **Purpose** |
|---|---|
| Python | Main programming language |
| YOLOv8n | Fruit object detection model |
| Ultralytics | YOLO model training and prediction |
| Google Colab | Model training and evaluation |

| | |
|---|---|
| Roboflow | Dataset annotation and correction |
| OpenCV | Image processing support |
| Streamlit | Web application development |
| Git | Version control |
| GitHub | Source-code and model storage |
| Streamlit Cloud | Online application deployment |

## 4. Dataset Preparation

The first stage involved preparing a dataset for training the fruit detection model. The dataset contained images belonging to three classes:

- Class 0: Apple
- Class 1: Banana
- Class 2: Orange

The dataset was organized into three main subsets:

- Training dataset: used to teach the model to recognize the fruit classes.
- Validation dataset: used to monitor model performance during training.
- Testing dataset: used to evaluate the trained model on images not used for training.

The dataset was configured using a data.yaml file containing the dataset paths and class names.

## 4.1 Dataset Annotation and Correction

The images required bounding-box annotations to identify the locations of fruits. During dataset inspection, some annotation files were found to be empty or invalid.

The annotations were reviewed and corrected using Roboflow. The corrected label files were then copied into the original dataset.

The dataset was subsequently checked for missing labels, empty annotation files, and invalid annotation entries. The final reported checks showed:

| Dataset split | Label files | Empty label files |
|---|---|---|
| Training | 211 | 0 |
| Validation | 46 | 0 |
| Testing | 43 | 0 |

Three placeholder files containing ANNOTATIONS_REQUIRED.txt were also identified and removed because they were not valid YOLO annotation files.

This correction was important because accurate annotations help the model learn the correct fruit classes and object locations.

## 5. Model Training Using YOLOv8n

The YOLOv8n model was selected because it is a lightweight object detection model suitable for applications that require relatively efficient inference.

The training process was conducted in Google Colab using the Ultralytics library. The original model checkpoint was loaded and fine-tuned using the corrected fruit dataset.

The main training configuration was:

| Parameter | Value |
|---|---|
| Model | YOLOv8n |
| Number of classes | 3 |
| Image size | 640 × 640 |
| Epochs requested | 30 |
| Batch size | 16 |
| Device | GPU (device=0) |
| Early stopping patience | 10 |
| Optimizer | Ultralytics default |

The model was trained to recognize apples, bananas, and oranges and to predict their locations using bounding boxes.

Training stopped early after 11 epochs because the early-stopping condition was reached. The best checkpoint was saved in the training output directory:

/content/fruit_training/yolov8n_fruit_corrected/weights/best.pt

The resulting model was downloaded and saved locally as best_corrected.pt for further testing and application integration.

## 6. Model Evaluation

After training, the model was evaluated using the test dataset. The evaluation considered precision, recall, mean Average Precision at 50% Intersection over Union (mAP50), and mean Average Precision across IoU thresholds from 50% to 95% (mAP50-95).

### 6.1 Overall Test Results

| Evaluation metric | Result |
|---|---|
| Precision | 0.7741 |
| Recall | 0.6951 |
| mAP50 | 0.8165 |
| mAP50-95 | 0.6389 |

These results indicate that the model achieved an overall mAP50 of approximately 81.65% on the test dataset.

### 6.2 Per-Class Test Results

| Fruit class | Precision | Recall | mAP50 | mAP50-95 |
|---|---|---|---|---|
| Apple | 0.8134 | 0.7667 | 0.8847 | 0.7324 |
| Banana | 0.6933 | 0.5143 | 0.6815 | 0.3947 |
| Orange | 0.8155 | 0.8043 | 0.8834 | 0.7895 |

The results show that the model performed relatively well in detecting apples and oranges. Banana detection was comparatively weaker, particularly in recall and mAP50-95.

This suggests that additional banana images, improved annotations, and more varied training examples could help improve the model.

### 7. Model Comparison

The original model and the corrected fine-tuned model were compared to assess the effect of correcting the dataset and retraining.

| Metric | Original model | Corrected model |
|---|---|---|
| Precision | 0.8140 | 0.7741 |
| Recall | 0.6997 | 0.6951 |
| mAP50 | 0.8072 | 0.8165 |
| mAP50-95 | 0.6737 | 0.6389 |

The corrected model achieved a slightly higher mAP50, increasing from approximately 80.72% to 81.65%. However, its precision and mAP50-95 were lower.

Therefore, the corrected model did not improve every evaluation metric. Both checkpoints were retained during development so their predictions could be compared.

## 8. Development of the Streamlit Application

After model training and evaluation, a web application was developed using Streamlit.

The application allows users to upload fruit images and run object detection using the trained YOLOv8n model.

The main application functions include:

1. Loading the trained YOLOv8 model.
2. Accepting an image uploaded by the user.
3. Allowing the user to adjust the detection confidence threshold.
4. Passing the uploaded image to the YOLO prediction function.
5. Displaying detected fruits and their bounding boxes.
6. Showing the predicted fruit classes and confidence scores.

The application uses the model checkpoint best.pt.

The prediction process uses the following configuration:

```
results = model.predict(
    image,
    conf=confidence,
    iou=0.45,
```

```
    verbose=False
)
```

The confidence parameter controls the minimum confidence required for a detection to be displayed. The IoU parameter is used during non-maximum suppression to manage overlapping predictions.

## 9. Integration with GitHub

GitHub was used to store and manage the application source code and model checkpoint.

The project repository was:

https://github.com/mugishalucky45-spec/fruit-detection-streamlit

The repository contains the main application file, model checkpoints, package configuration, and README documentation.

During integration, an incorrect or corrupted model checkpoint caused a loading error in Streamlit Cloud. The application reported that best.pt was not a valid or loadable checkpoint.

To resolve the problem, the verified local model checkpoint was copied into the required filename, best.pt, and uploaded through Git Bash using Git.

The local and remote repositories initially had unrelated commit histories. The histories were merged, the verified local checkpoint was selected to resolve the binary-file conflict, and the merge was committed successfully.

The final push to GitHub was successful:

970ced7

This confirmed that the updated application and verified model checkpoint were uploaded to the intended repository.

## 10. Deployment Using Streamlit Cloud

Streamlit Cloud was used to make the fruit detection application accessible through a web browser.

The deployment process involved:

1. Connecting the GitHub repository to Streamlit Cloud.
2. Selecting the application entry file, app.py.
3. Installing the required Python packages.
4. Loading the trained YOLOv8n checkpoint.

5. Deploying the application.
6. Opening the deployed application and testing its functionality.

Initially, deployment encountered a model-file error because the checkpoint was corrupted or did not match the filename expected by the application.

After uploading the verified model file and updating the application to load best.pt, deployment completed successfully.

The application was subsequently opened and confirmed to be working.

## 11. Testing and Observations

The model was tested using individual-fruit images and images containing multiple fruit classes.

The testing process revealed that the model could detect fruits and draw bounding boxes around recognized objects. However, its performance varied depending on the image and the combination of fruits present.

Some mixed-fruit images produced correct detections, while others showed incorrect classifications, missing fruit detections, or low-confidence predictions.

For example, in one mixed-fruit test, the model correctly detected an apple with a confidence score of approximately 0.89, but incorrectly classified an orange as an apple with a confidence score of approximately 0.695. A banana prediction appeared with a much lower confidence score.

This demonstrated that changing the confidence threshold alone could not solve all classification errors. The model's learned features and the quality and diversity of its training data also influence prediction accuracy.

## 12. Challenges Encountered and Solutions

| **Challenge** | **Solution** |
|---|---|
| Empty annotation files | Reviewed and corrected annotations using Roboflow |
| Invalid placeholder label files | Removed the three invalid files |
| Model performance differences between fruit classes | Evaluated each class separately and identified banana detection as an area for improvement |
| Incorrect predictions in mixed-fruit images | Conducted additional testing and identified the need for more representative training data |

| | |
|---|---|
| Corrupted model checkpoint on Streamlit Cloud | Replaced it with the verified local model checkpoint |
| Application loading the wrong model filename | Updated the model loader to use best.pt |
| GitHub merge conflict involving the binary model file | Selected the verified local checkpoint and completed the merge |
| Updating the intended GitHub repository | Corrected the Git remote to point to fruit-detection-streamlit |

## 13. Limitations of the System

Although the application is operational, several limitations remain:

1. The model does not detect every fruit correctly in all mixed-fruit images.
2. Banana detection performance is lower than apple and orange detection performance.
3. Some objects may be missed when fruits overlap or appear in unusual positions.
4. Predictions may vary with image quality, lighting, background, and fruit size.
5. The model's test performance does not guarantee the same accuracy on all real-world images.
6. The system currently depends on uploaded images rather than a continuous live camera stream.

These limitations indicate that further training and testing are required before using the system in a real-world commercial environment.

## 14. Recommendations for Future Improvement

The following improvements are recommended:

- Increase the number and diversity of banana training images.
- Include more images containing different combinations of apples, bananas, and oranges.
- Improve annotation consistency and bounding-box accuracy.
- Apply data augmentation to improve performance under different lighting conditions and backgrounds.
- Compare the original and corrected checkpoints using the same mixed-fruit test images.

- Tune confidence and IoU thresholds using a consistent evaluation procedure.
- Test the application with more real-world images.
- Consider adding live camera detection in a future version.
- Monitor inference speed and resource usage if the application is later deployed on an edge device.

## 15. Conclusion

This project successfully developed and deployed a fruit detection application using the YOLOv8n object detection algorithm and Streamlit.

The work involved preparing and correcting the fruit dataset, fine-tuning the YOLOv8n model, evaluating its performance, developing a web application, integrating the trained model with GitHub, and deploying the application through Streamlit Cloud.

The corrected model achieved an overall test precision of approximately 77.41%, recall of 69.51%, mAP50 of 81.65%, and mAP50-95 of 63.89%.

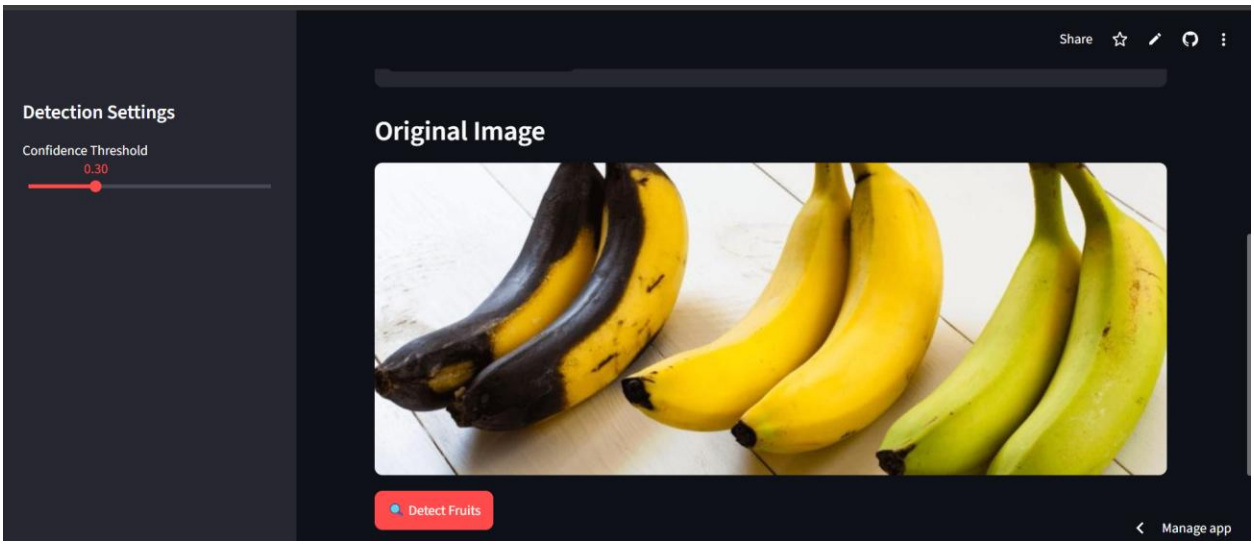
The final application successfully loads the verified best.pt checkpoint and allows users to upload images for fruit detection.

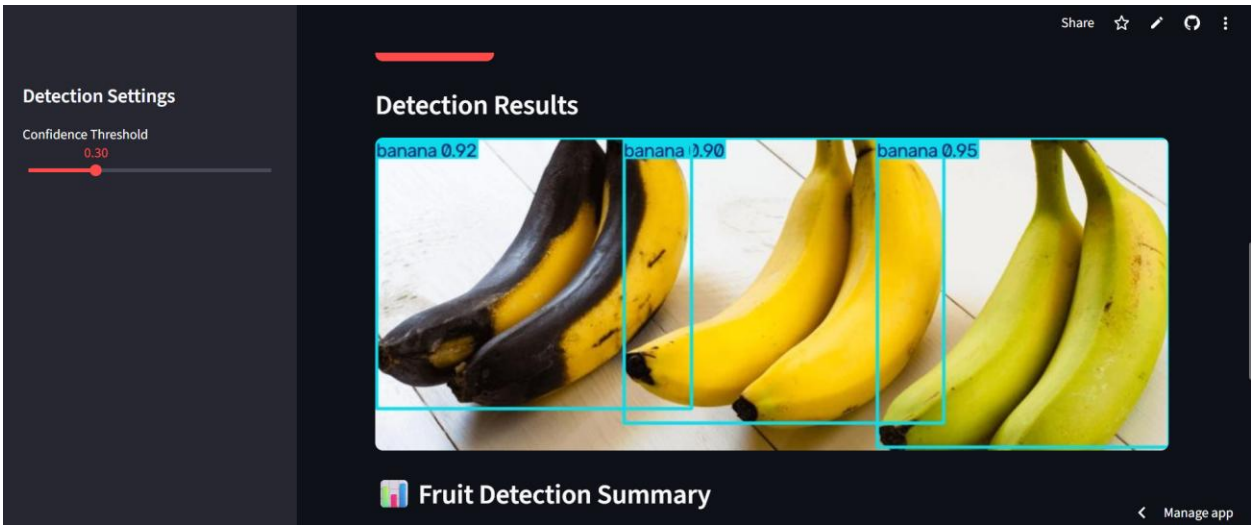
Despite the successful deployment, some classification errors remain, especially in mixed-fruit images and banana detection. Further dataset improvement, model training, and testing are recommended to improve reliability.

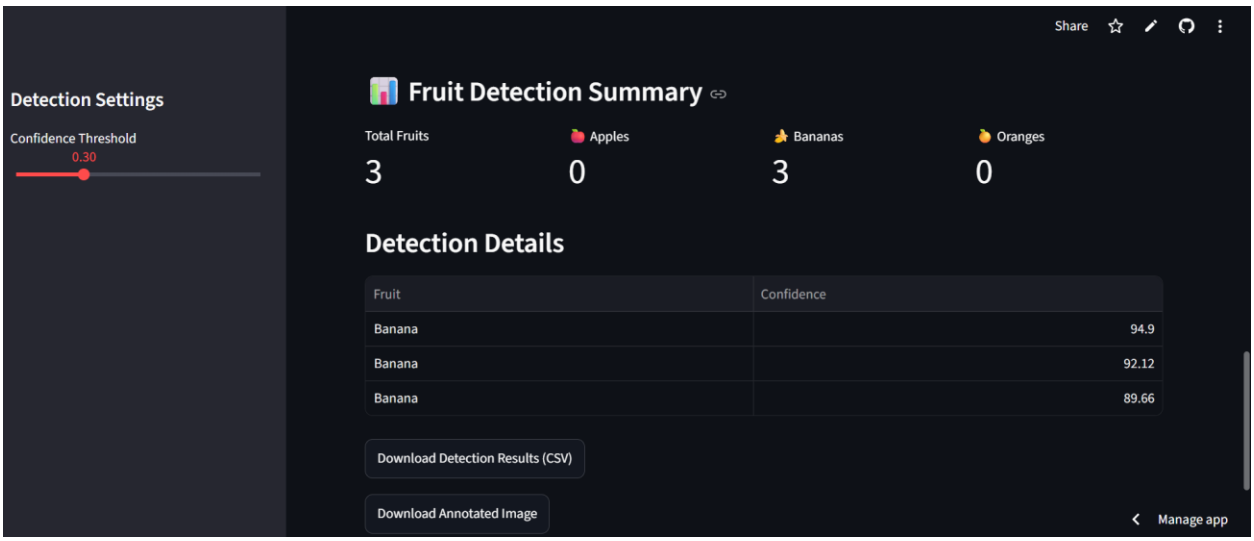
Overall, the project demonstrates how deep learning and web application technologies can be combined to develop an accessible fruit detection system.

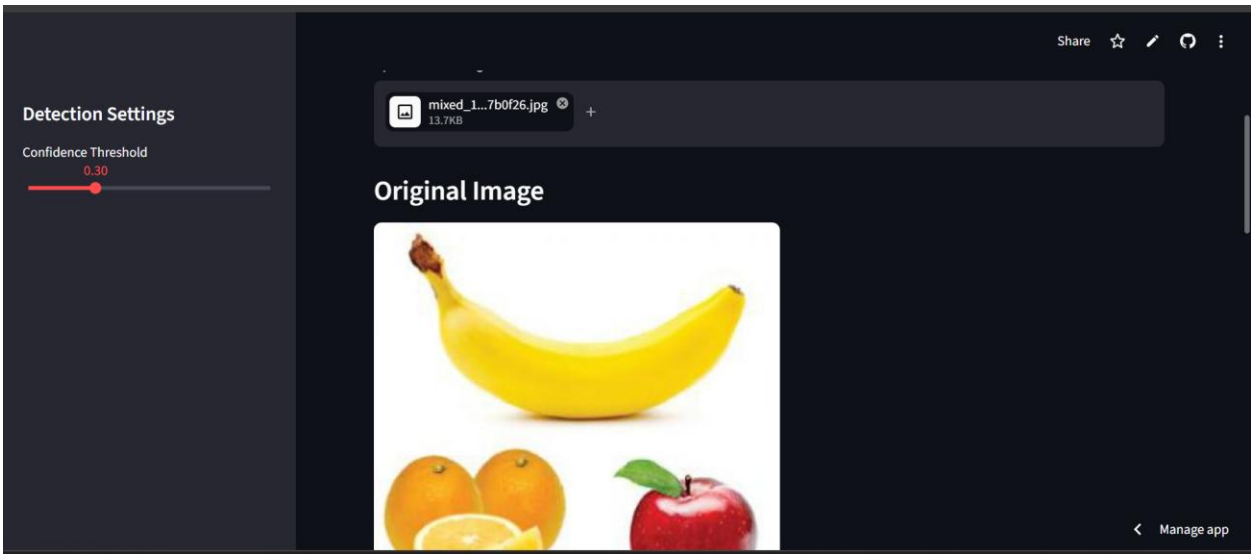
## 16. Test results

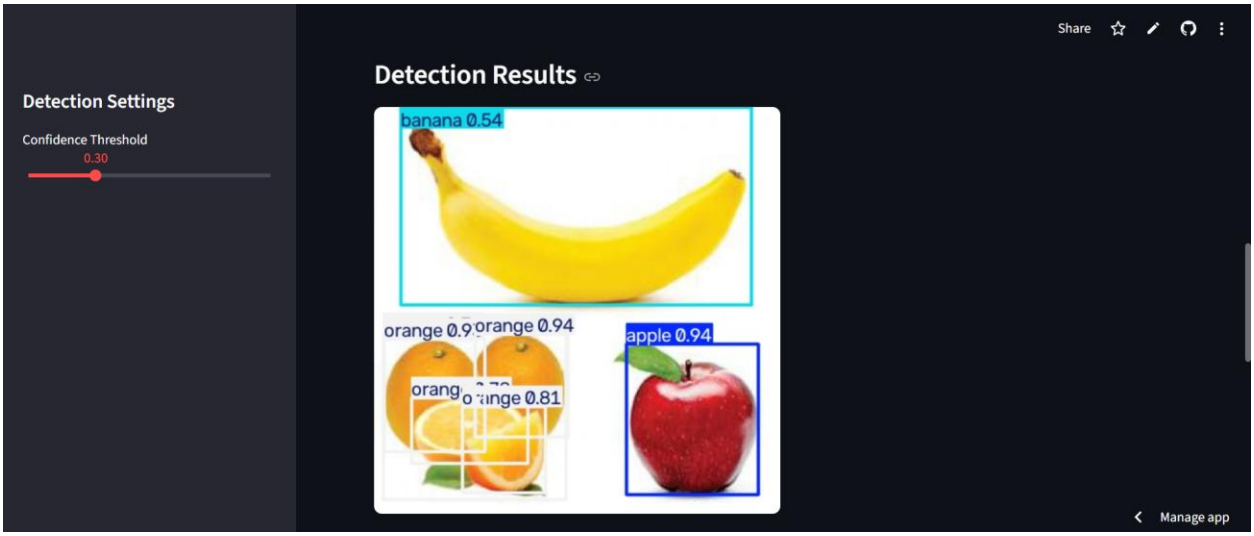



Share
Detection Settings
Confidence Threshold
0.30
Original Image
Detect Fruits
Manage app


Share
Detection Settings
Confidence Threshold
0.30
Detection Results
banana 0.92
banana 0.90
banana 0.95
Fruit Detection Summary
Manage app


Share
Detection Settings
Confidence Threshold
0.30
Fruit Detection Summary
Total Fruits
3
Apples
0
Bananas
3
Oranges
0
Detection Details
Fruit
Confidence
Banana
94.9
Banana
92.12
Banana
89.66
Download Detection Results (CSV)
Download Annotated Image
Manage app




## 17. References


1. Ultralytics. *YOLO Documentation*. https://docs.ultralytics.com/
2. Google. *Google Colaboratory*. https://colab.research.google.com/
3. Streamlit. *Streamlit Documentation*. https://docs.streamlit.io/
4. GitHub. *GitHub Documentation*. https://docs.github.com/
5. Roboflow. *Roboflow Documentation*. https://docs.roboflow.com/


**Project repository:** https://github.com/mugishalucky45-spec/fruit-detection-streamlit

**Application project: https://fruit-detection-app-lxv4cm6hyhndsje8gnuzl8.streamlit.app/**